



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG VỚI UML

BỘ MÔN CÔNG NGHỆ PHẦN MỀM



Giới thiệu nội dung học phần

Bài 1: Giới thiệu quy trình ICONIX Process

Bài 2: Mô hình hóa miền (Domain modeling)

Bài 3: Mô hình hóa Use Case (Use Case Modeling)

Bài 4: Requirements Review – Kiểm duyệt yêu cầu → **Kiểm tra TX1**

Bài 5: Robustness Analysis — Cầu nối từ Use Case đến Thiết kế

Bài 6: Preliminary Design Review — Kiểm duyệt thiết kế sơ bộ → **Kiểm tra TX 2**

Bài 7: Technical Architecture — Kiến trúc kỹ thuật

Bài 8: Sequence Diagrams

Bài 9: Critical Design Review (CDR) → **Kiểm tra TX3**



TẦM QUAN TRỌNG CỦA MÔN HỌC

Môn học này có quan trọng không → Có. Vì:

- Nó là tài liệu bắt buộc trong đề tài, đồ án, dự án...
- Nó là cơ sở để cho các lập trình viên viết code
- Nó là tài liệu để cho các mô hình AI hiện đại sinh mã nguồn, chương trình...

BÀI 3: USE CASE MODELING

Về kiến thức

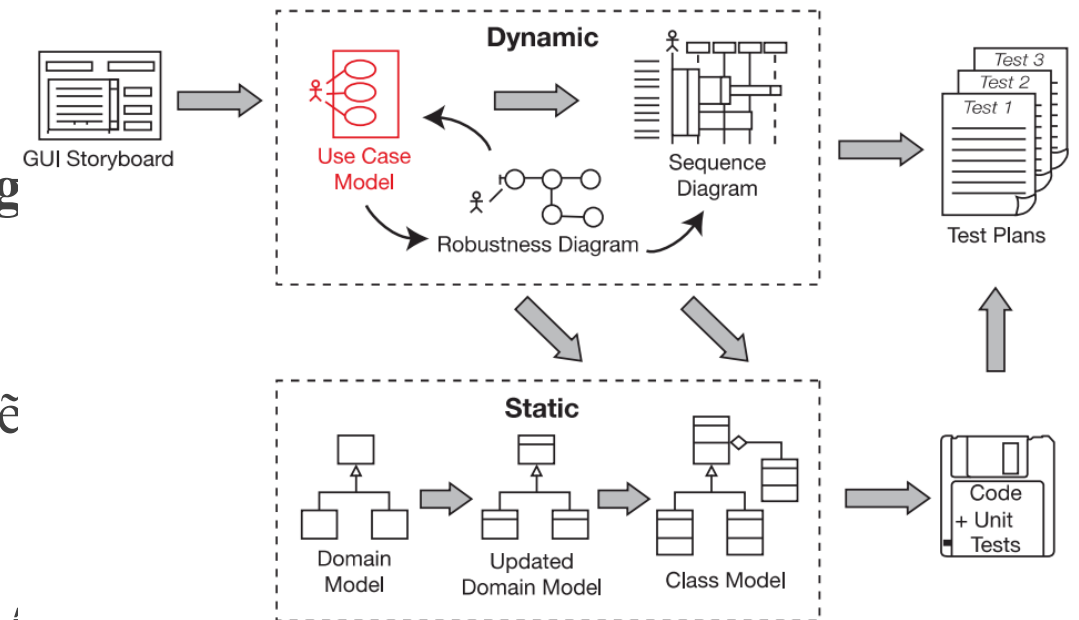
- 🗺️ Cầu nối giữa yêu cầu và thiết kế phần mềm
- 🗣️ Chuyển ngôn ngữ tự nhiên thành kịch bản hành vi có cấu trúc
- 🧩 Gắn kết với Domain Model để đảm bảo nhất quán
- 📦 Tổ chức Use Case thành các gói chức năng rõ ràng

Về kỹ năng

- ✍️ Viết được Use Case ngắn gọn, rõ ràng, có luồng chính & ngoại lệ
- 🧠 Xác định được Actor và quan hệ với hệ thống
- ✨ Biểu diễn Use Case bằng sơ đồ chuẩn UML
- 🔗 Liên kết Use Case với Domain Model để đảm bảo tính nhất quán

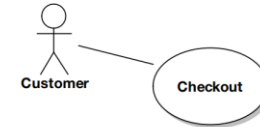
BỐI CẢNH VÀ VAI TRÒ

- 📖 **Domain Model** → mô tả cấu trúc tĩnh của hệ thống
- 👥 Phần mềm phải phản ánh **hành vi & tương tác người dùng**
- 📄 **Use Case Modeling** trả lời: “Người dùng sẽ làm gì với hệ thống?”
- ⚡ Cầu nối từ **mô hình tĩnh** → **hành vi** → **thiết kế**



Cầu nối giữa mô hình tĩnh & hành vi động

- 🧭 **Domain Model** → định nghĩa “vật thể”
- 😊😞 **Use Case** → định nghĩa “hành động”
- 🔗 **Kết hợp** → nền tảng cho thiết kế & code
- 📄 **Domain** = nhân vật & bối cảnh | **Use Case** = kịch bản hành động



Viết Use Case trong ngữ cảnh Domain Model

-  Use Case gắn chặt với Domain Model → Sử dụng thuật ngữ từ Domain Model
-  Giả định Domain Model ban đầu “chưa đúng”
-  Viết Use Case để phát hiện lỗi hỏng mô hình
-  Cập nhật Domain Model → chính xác & thực dụng hơn



Vai trò trung tâm trong ICONIX

 Use Case = “dòng máu” của quy trình

 Là đầu vào cho Robustness, Sequence, Coding

 Use Case mơ hồ → thiết kế lung lay

  Rõ ràng, đủ luồng chính & lỗi → tiết kiệm, nhất quán



Ngôn ngữ tự nhiên → Use Case

-  Yêu cầu tự nhiên → mơ hồ, đa nghĩa
-  Use Case = công cụ chuyển đổi có hệ thống
-  3 bước chính:
 - **1** Xác định tác nhân & hành vi
 - **2** Tách Use Case sơ bộ
 - **3** Viết kịch bản chi tiết

Viết kịch bản như kể chuyện

-  Tránh liệt kê chức năng khô khan
-  Viết như mini-story giữa actor ↔ hệ thống
-  Tập trung vào hành vi & phản hồi, không GUI
-  Dễ đọc – linh hoạt – tái sử dụng đa nền tảng



Đừng quên “Rainy-day Scenarios” ☁️

- 🌟 Nhiều nhóm chỉ viết luồng thành công
- ⚡ Nhưng lỗi thường lộ ra ở tình huống “trời mưa”
- 🧭 Rainy-day giúp phát hiện lỗ hổng sớm
- 🧠 Là cơ chế kiểm thử mô hình ngay từ đầu

Gắn kịch bản với Domain Model

✓ Mỗi bước trong Use Case phải gắn với lớp miền hoặc quan hệ cụ thể.

🧠 Giúp phát hiện khái niệm còn thiếu hoặc chưa rõ trong Domain Model.

⊘ Tránh viết kịch bản bị trôi sang mô tả GUI hay hành vi “vô căn cứ”.

↻ Viết Use Case và cập nhật Domain Model là quá trình lặp liên tục.

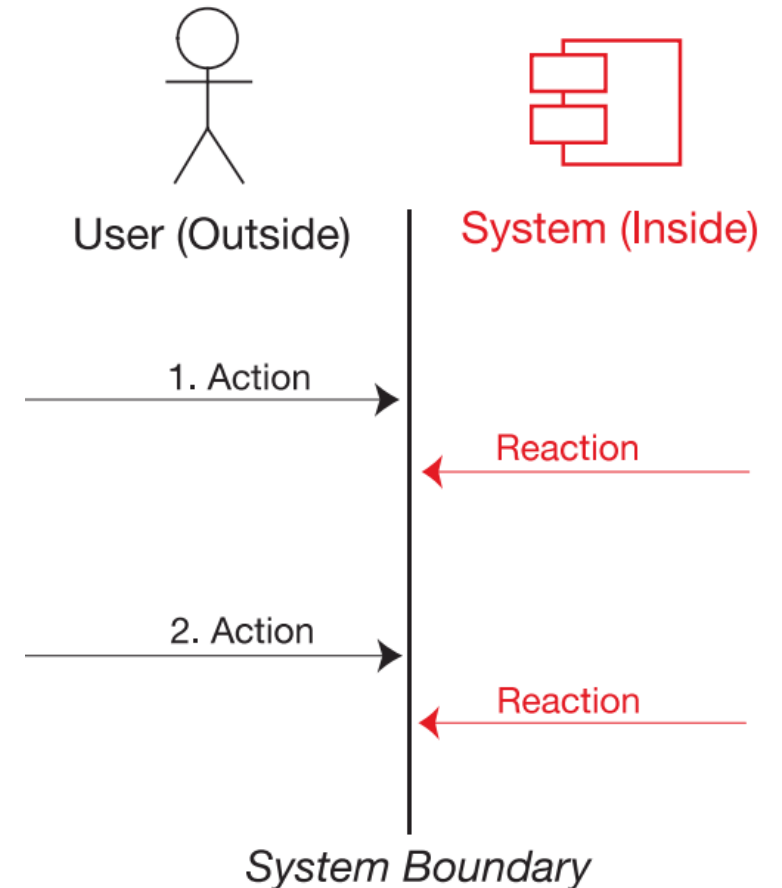
Kiểm duyệt Use Case sớm

- 🔍 **Review sớm** = phát hiện lỗi logic & thiếu sót trước khi thiết kế.
- 🧠 Tất cả các bên cùng kiểm tra kịch bản & tình huống biên.
- 💡 Lỗi phát hiện sớm → chi phí sửa thấp, tránh “chữa cháy” về sau.
- 🧭 Use Case rõ ràng = nền tảng vững cho Robustness → Sequence → Code.

Top 10 Use Case Modeling Guidelines

Một số nguyên tắc cốt lõi:

- 1 Viêt bằng ngôn ngữ tự nhiên, dễ hiểu
- 2 Luôn trong ngữ cảnh Domain Model
- 3 Giữ ở mức khái niệm, tránh kỹ thuật
- 4 Xác định rõ actor chính & phụ
- 5 Bắt đầu từ luồng chính → thêm luồng phụ
- 6 Đừng quên rainy-day scenarios ☁️
- 7 Độ dài vừa phải, không quá ngắn/dài
- 8 Không thay use case bằng sơ đồ
- 9 Review nghiêm túc như checkpoint chính
- 10 Chấp nhận & tinh chỉnh qua nhiều vòng lặp



Organizing Use Cases into Packages

-  Khi số lượng Use Case tăng → quản lý trở nên phức tạp
-  Dễ rơi vào tình trạng trùng lặp, mâu thuẫn, “mớ hỗn độn”
-  **Nhóm Use Case theo Package** giúp tổ chức & định hướng rõ ràng
-  Package phản ánh cấu trúc chức năng tự nhiên của hệ thống

general

- + Add to Wish List
- + Cancel Order
- + Edit Shopping Cart
- + Login
- + Logout
- + Open an Account
- + Return a book
- + View Order History
- + Where's My Stuff?

shopping

- + Customer
- + Add Item to Shopping Cart
- + Checkout
- + Edit Shopping Cart
- + Enter Address
- + Pay by Card
- + Pay by Check
- + Pay by Purchase Order
- + Remove Item From Shopping Cart
- + View Recommendations
- + View Review
- + Write Reader Review

admin

- + Customer Service
- + Seller
- + Shipping Clerk
- + Webmaster
- + Add Books to Catalog
- + Add Editorial Review
- + Add External Books to Catalog
- + Dispatch Order
- + Moderate Customer Reviews
- + Monitor Stock Levels
- + Order Books from Publisher
- + Process Refund
- + Remove Books from Catalog
- + Remove External Books from Catalog
- + Respond to Enquiry
- + Unlock Locked Account

searching

- + Advanced Search
- + Search by Author
- + Search by Category
- + Search by Keyword
- + Search by Title
- + Search for Books



Từ Domain Model → Use Case Package

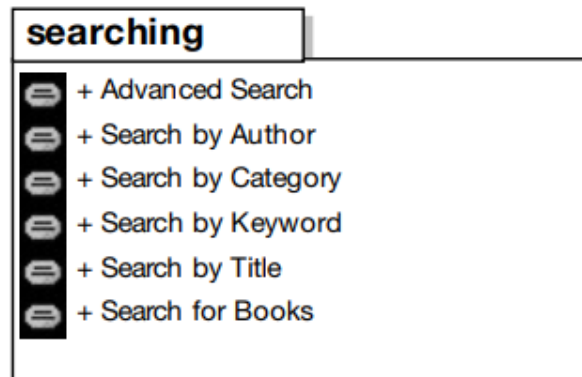
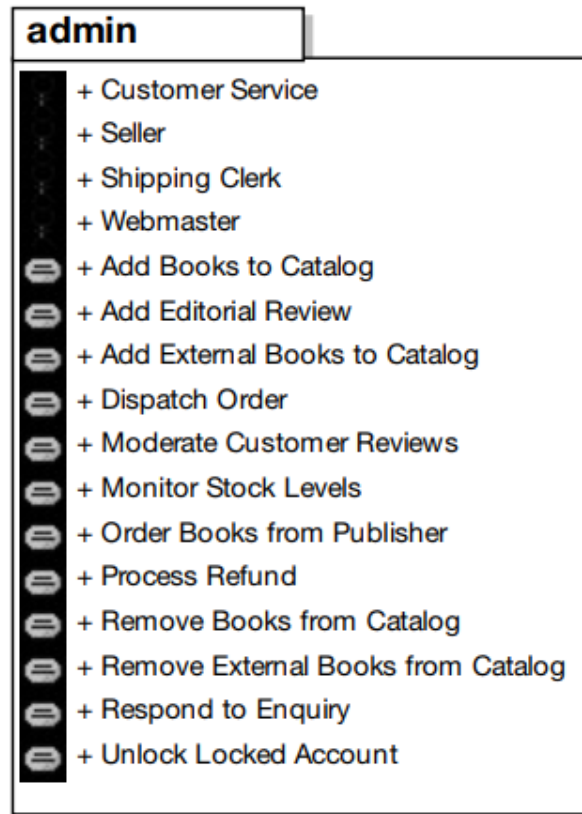
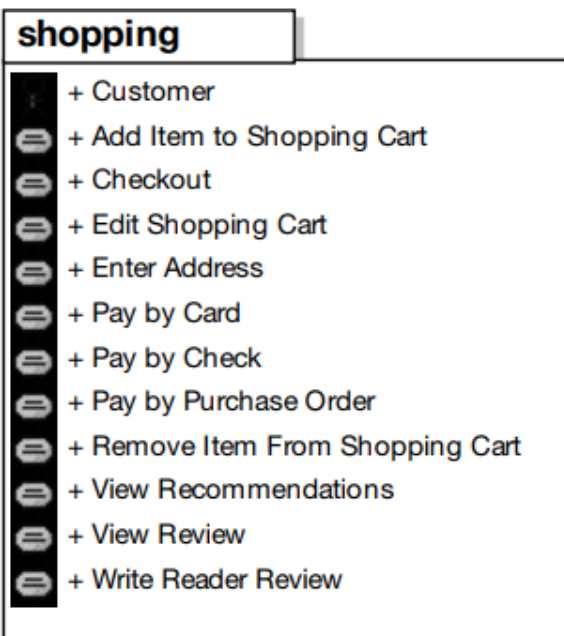
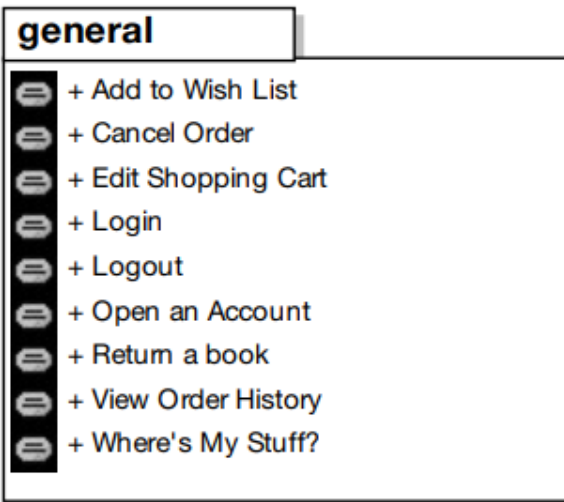


-  **Domain Model** = bản đồ khái niệm tĩnh của hệ thống
-  Nhóm các lớp có quan hệ tự nhiên → thành “cụm chức năng”
-  Tổ chức Use Case theo các cụm này, **không theo GUI**
-  Package phản ánh **cấu trúc nghiệp vụ thật sự**



Package = Ranh giới chức năng thật sự

-   Đừng nhóm theo **menu** / **GUI** (Main Menu, Admin Menu...)
-  GUI thay đổi → cách nhóm sẽ lỗi thời nhanh chóng
-   Nhóm theo **functional responsibility** (trách nhiệm nghiệp vụ)
-  Package → ánh xạ tốt sang module backend, bền vững khi UI đổi



Từ Package → Use Case Diagram

 Mỗi **package** → vùng chứa nhóm use case liên quan

 Các **actor** kết nối tới use case trong package

 Cung cấp cái nhìn **top–down** toàn hệ thống

 Dễ review kiến trúc & giới thiệu cho người mới

Internet Bookstore - Edit Shopping Cart

Items in Your Shopping Cart	Price:	Qty:
Domain Driven Design -----	\$42.65	1
Extreme Programming Refactored -----	\$29.65	1
Update		

Viết Use Case ở thể chủ động

-  “Actor thực hiện hành động, hệ thống phản hồi”
-  Rõ ràng, mạch lạc, dễ kiểm chứng
-  Tránh câu bị động mơ hồ (“hệ thống được phép...”)
-  Giúp phát hiện hành vi, lớp miền và luồng lỗi nhanh hơn

BASIC COURSE:

The Customer clicks the Write Review button for the book currently being viewed, and the system shows the Write Review screen. The Customer types in a Book Review, gives it a Book Rating out of five stars, and clicks the Send button. The system ensures that the Book Review isn't too long or short, and that the Book Rating is within one and five stars. The system then displays a confirmation screen, and the review is sent to a Moderator, ready to be added.

ALTERNATE COURSES:

User not logged in: The user is first taken to the Login screen and then to the Write Review screen once he is logged in.

The user enters a review that is too long (text > 1MB): The system rejects the review and responds with a message explaining why the review was rejected.

The review is too short (< 10 characters): The system rejects the review.



BÀI TẬP

Viết đặc tả Use case cho các chức năng

- Hiển thị sản phẩm
- Thêm mới sản phẩm
- Xóa sản phẩm
- Cập nhật sản phẩm
- Tìm kiếm sản phẩm theo tên
- Hoàn thiện sơ đồ miền + đặc tả use case để chuẩn bị cho bài kiểm tra TX 1